

EXPLORING HIDDEN MARKOV MODELS IN ISOLATED WORD SPEECH RECOGNITION

Erfan Mirhosseini, Rishab Jha, Shervin Sajan, Vladimir Borozenets
Yanshu Zhao, Ziyu Wang

EEEM030 Group Coursework 2
Speech Recognition



*School of Computer Science and Electrical and Electronic
Engineering*

Faculty of Engineering and Physical Sciences

University of Surrey

Guildford, Surrey, GU2 7XH, UK

December 2024

Professor: Prof Philip Jackson

Group 7

Abstract

Speech recognition systems have long relied on Hidden Markov Models (HMMs) for their ability to capture the sequential and probabilistic nature of speech signals. By modeling temporal dynamics through state transitions, HMMs enable the mapping of acoustic features to lexical units, forming a foundational approach to speech recognition. This study presents an isolated word recognition system designed for a constrained vocabulary of eleven words, leveraging HMMs to model speech patterns. Mel-Frequency Cepstral Coefficients (MFCCs) serve as the primary acoustic features, enabling robust spectral characterization of spoken utterances. Training was performed using the Baum-Welch algorithm, and decoding is performed using the Viterbi algorithm to determine the most likely sequence of hidden states. The system achieves a test accuracy of 53.64%, reflecting the complexity of differentiating among phonetically similar words. The resulting performance offers insight into the relationship between acoustic feature variability and lexical classification accuracy within limited-vocabulary speech recognition tasks.

Contents

1	INTRODUCTION	4
1.1	Overview	4
1.2	Objectives	4
1.3	Team Structure	4
2	Feature Extraction	6
2.1	Essentials of MFCCs	6
2.2	Acoustic Feature Representation: MFCCs	6
2.3	Data Normalization and Padding	7
3	Training with Hidden Markov Models	9
3.1	Model Parameterization	9
3.2	Initialization	9
3.3	Forward-Backward Algorithms	10
3.4	Baum-Welch Re-Estimation	10
4	Decoding with the Viterbi Algorithm	12
4.1	Objective	12
4.2	Algorithm	12
4.3	Implementation Considerations	12
4.4	Practical Considerations and Extensions	12
5	Results	14
5.1	Training and Validation Results	14
5.2	Test Results	18
5.3	Error Analysis	22
6	Conclusion	25

1 INTRODUCTION

1.1 Overview

Hidden Markov Models (HMMs) are probabilistic models widely used in speech recognition due to their ability to model sequential data effectively. They represent speech as a sequence of hidden states, corresponding to linguistic or acoustic units such as phonemes, and observable outputs, such as acoustic features extracted from speech signals. The core assumption of HMMs is that each state depends only on its predecessor and that observations are conditionally independent given the current state.[1]

HMMs are used to infer a sequence of linguistic units from the acoustic features of speech. These models use probabilistic transitions between hidden states to capture the temporal dynamics of speech. For instance, a phoneme may transition to another phoneme or word in a probabilistic manner, reflecting natural language patterns. The emission probabilities in an HMM model determine how likely a particular acoustic feature is for each state, enabling the system to handle variability in pronunciation, speaking rate, and background noise.

By integrating acoustic and language models, HMMs can effectively capture both the variability in speech signals and linguistic structure, bridging the gap between raw acoustic data and meaningful text. Since their introduction in the 1980s, HMMs have been the core of early speech recognition systems and remain integral to many contemporary models[2], providing a foundational framework for transforming spoken language into text.

1.2 Objectives

The main objective of this report is to develop an isolated word recognition system using Hidden Markov Models for a vocabulary of eleven words. The project aims to implement key machine learning algorithms for feature extraction, model initialisation, training, and evaluation of the developed speech recognition systems.

The primary goals include extracting Mel-frequency cepstral coefficients (MFCCs) from audio data, initialising HMMs with the appropriate parameters, and training these models using the Baum-Welch algorithm. The system's performance will be then assessed by implementing the Viterbi algorithm for decoding and evaluating recognition accuracy.

1.3 Team Structure

The task distribution within the team was organised based on each member's strengths and expertise in Python and machine learning. This approach ensured that everyone could contribute effectively to the project. Here's the group contributions (The numbers listed in the contributions do not indicate the level of effort or ranking):

1. Yanshu Zhao

- Collaborated with Erfan Mirhosseini on implementing the HMM coding pipeline, including training and algorithms.
- Wrote the *Training* section.
- Edited the report.

2. Seyederfan Mirhosseini

- Collaborated with Yanshu Zhao on implementing the HMM coding pipeline.
- Worked on re-estimation and fine-tuning.
- Wrote the *Abstract*, *Viterbi* and *Conclusion* sections.
- Edited and proofread the report.

3. Vladimir Borozents

- Handled data preprocessing, including normalization and padding.
- Wrote the *Feature Extraction* section.

4. Shervin Sajan

- Assisted with data preprocessing, including feature extraction.
- Wrote the *Introduction* section and *Essentials of MFCCs* subsection.

5. Ziyu Wang

- Analyzed training and validation results.
- Collaborated on error analysis with Rishabh Jha.
- Wrote the *Training Results* subsection.

6. Rishabh Jha

- Analyzed test results.
- Collaborated on error analysis with Ziyu Wang.
- Wrote the *Test Results* subsection.
- Edited the report.

2 Feature Extraction

2.1 Essentials of MFCCs

Mel-Frequency Cepstral Coefficients are a vital part in speech signal processing and feature extraction, widely recognised for its ability to capture critical information about the vocal tract. In the source-filter model of speech production, the vocal tract acts as a filter, shaping the raw sound produced by the vocal cords. MFCCs effectively represent this filter by capturing its spectral characteristics in a compact and meaningful set of coefficients[3]. The process begins with the Fourier transformation of the speech signal to analyse its frequency content. This is followed by applying a perceptually inspired Mel-spaced filter bank, which mimics the nonlinear frequency resolution of the human ear, focusing on frequencies most relevant to human speech perception[4]. The resulting frequency spectrum is then transformed into the cepstral domain, producing coefficients that succinctly describe the vocal tract's shape and behaviour during speech. MFCCs are integral to modern speech recognition systems because they provide a robust, compact representation of speech that is less sensitive to variations such as pitch and speaker identity[3]. By compressing essential vocal tract information into a manageable number of coefficients, MFCCs have become the dominant method for spectral feature extraction since their introduction.

2.2 Acoustic Feature Representation: MFCCs

To perform speech recognition, we begin by extracting meaningful acoustic features from raw audio signals. Mel-Frequency Cepstral Coefficients (MFCCs) are a standard choice for this purpose. They capture spectral properties of the audio signal in a way that approximates the human auditory perception of pitch and loudness, using a non-linear Mel scale.

The MFCC extraction process involves several steps:

1. **Pre-Emphasis:** The raw audio signal is first pre-emphasized to boost high-frequency components and improve the signal-to-noise ratio.
2. **Frame Segmentation:** The continuous signal is divided into short, overlapping frames. Here, we choose a 30 ms frame length with a 10 ms hop (overlap) to ensure quasi-stationary analysis and smooth transitions between frames.
3. **Windowing:** Each frame is multiplied by a window function (often a Hamming window) to reduce discontinuities at the frame edges.
4. **Fourier Transform:** The Discrete Fourier Transform (DFT) is applied to obtain the magnitude spectrum of each frame.
5. **Mel-Scale Filtering:** A set of triangular Mel filters is applied to the magnitude spectrum to emphasize perceptually important frequencies. The Mel scale, $M(f)$, is defined as:

$$M(f) = 2595 \log_{10} \left(1 + \frac{f}{700} \right), \quad (1)$$

where f is the frequency in Hertz.

6. **Logarithm and DCT:** Taking the logarithm of the filtered spectrum and applying the Discrete Cosine Transform (DCT) yields the MFCCs, which are the final acoustic features.

In our implementation, we use the `librosa` library[5] to compute a fixed number of $D = 13$ MFCCs per frame. These 13 coefficients have been shown to be effective for many speech recognition tasks, although this number may be adjusted depending on the complexity of the vocabulary and the nature of the audio data.

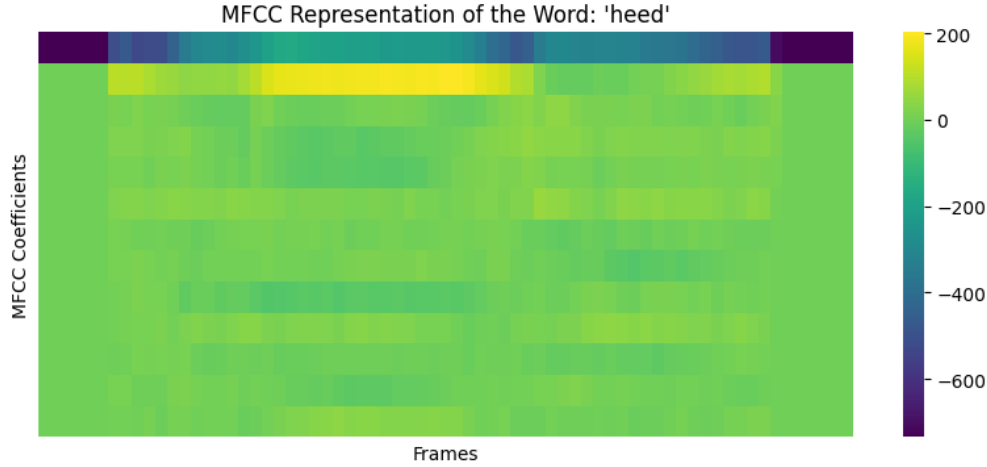


Figure 1: An example MFCC representation of the word “heed.” The horizontal axis represents time (frames), and the vertical axis represents the MFCC coefficients. The color scale indicates the magnitude of each coefficient.

Figure 1 provides an example visualization of the MFCC representation for the spoken word “heed.” Each column corresponds to a time frame, while each row corresponds to one MFCC coefficient. The color intensity reflects the magnitude of these coefficients, allowing us to visually inspect the spectral characteristics over time.

Figure 2 illustrates the high-level overview of our encoding function, which takes an array of speech recordings and corresponding hyperparameters (such as frame length and hop size) as input, and produces a tensor of MFCC feature matrices. Each matrix corresponds to one recording, with rows representing frames and columns representing MFCC coefficients.

2.3 Data Normalization and Padding

Before extracting MFCCs, the audio signals are normalized so that their amplitude lies within $[-1, 1]$. Trimming of leading and trailing silences is performed beforehand. After MFCC extraction, we ensure uniform input length by zero-padding all feature matrices to a common maximum

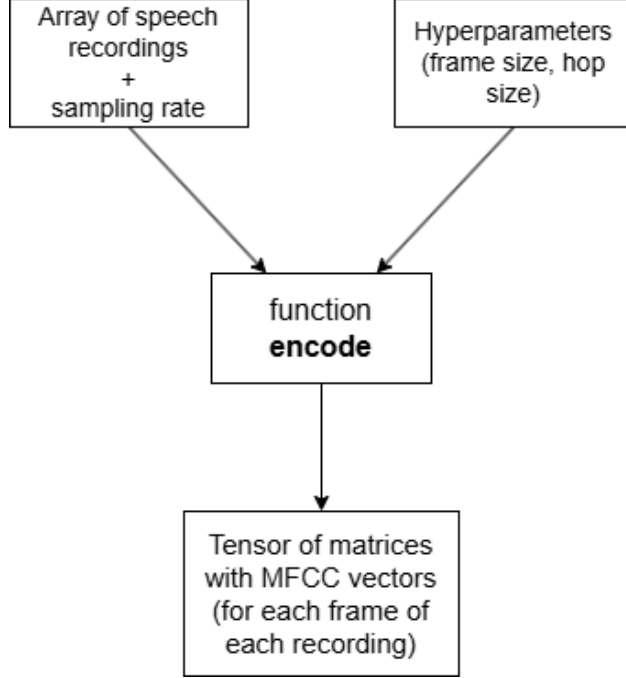


Figure 2: A schematic representation of the encoding process that converts raw speech signals and hyperparameters into a tensor of MFCC feature matrices.

frame length $F_{\max}$. This produces a dataset tensor:

$$\mathbf{X} \in \mathbb{R}^{N \times F_{\max} \times D},$$

where N is the number of utterances, $F_{\max}$ is the maximum number of frames across the dataset, and $D = 13$ is the number of MFCC coefficients per frame.

We also maintain a corresponding mask $\mathbf{M} \in \{\text{True}, \text{False}\}^{N \times F_{\max}}$ to distinguish between actual and padded frames. The mask is crucial during training and decoding steps to prevent computations over non-existent (padded) frames.

3 Training with Hidden Markov Models

3.1 Model Parameterization

We model each word as a Hidden Markov Model (HMM) with a left-to-right topology. This structure reflects the sequential progression of phonemes in a word. The model comprises:

- N states (including $N - 1$ emitting states and an absorbing final state).
- A transition matrix $\mathbf{A} \in \mathbb{R}^{(N+1) \times (N+1)}$, where $(N + 1)$ denotes an absorbing end state.
- Emission probability distributions $\mathbf{B}$, each modeled as a multivariate Gaussian distribution over the MFCC feature space.
- An initial state distribution π .

The states are arranged such that transitions occur either from a state to itself or to the next state. This “left-to-right” topology prevents backward transitions and mimics the temporal flow of speech production. Transition probabilities are initialized to represent average durations for phonemes, while emission distributions capture variations in acoustic features for each state.

3.2 Initialization

Initial State Distribution: We assign a probability of 1 to the first state and 0 to all others:

$$\pi_i = \begin{cases} 1 & \text{if } i = 1, \\ 0 & \text{otherwise.} \end{cases}$$

Transition Matrix Initialization: To reflect average state durations, we define τ as the average frame duration per state. We initialize:

$$A_{ij} = \begin{cases} \exp\left(-\frac{1}{\tau-1}\right), & j = i, \\ 1 - \exp\left(-\frac{1}{\tau-1}\right), & j = i + 1, \\ 0, & \text{otherwise.} \end{cases}$$

The last (absorbing) state transitions only to itself with probability 1.

Emission Model Initialization: The emission distributions $\mathbf{B}$ are initialized using the global mean and covariance of all training data. For each emitting state i , the mean vector μ_i is initially set to the global mean, and the covariance matrix Σ_i is set to a diagonal covariance derived from the global covariance. A small regularization constant $\epsilon = 10^{-6}$ is added to the covariance diagonal to ensure numerical stability.

3.3 Forward-Backward Algorithms

To train the HMM, we first need to compute the likelihood of observation sequences under the current model. We use the forward-backward algorithm for this purpose, carefully implementing computations in the log-domain to avoid underflow.

Forward Probabilities $\alpha_t(i)$: For a sequence of observations $\{O_1, O_2, \dots, O_T\}$:

$$\alpha_t(i) = \left(\sum_j \alpha_{t-1}(j) A_{ji} \right) B_i(O_t).$$

We compute this in the log-domain:

$$\log \alpha_t(i) = \log \sum_j \exp(\log \alpha_{t-1}(j) + \log A_{ji} + \log B_i(O_t)).$$

Backward Probabilities $\beta_t(i)$: Similarly, the backward probabilities are:

$$\beta_t(i) = \sum_j A_{ij} B_j(O_{t+1}) \beta_{t+1}(j),$$

also computed in the log domain.

The combination of forward and backward probabilities yields the posterior state occupation probabilities and provides the necessary statistics for parameter re-estimation.

3.4 Baum-Welch Re-Estimation

The Baum-Welch algorithm is an Expectation-Maximization (EM) algorithm used to iteratively update the parameters $\{\mathbf{A}, \mathbf{B}, \boldsymbol{\pi}\}$ to locally maximize the likelihood of the training data.

E-Step: We compute:

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{\sum_k \alpha_T(k)}, \quad \xi_t(i, j) = \frac{\alpha_t(i) A_{ij} B_j(O_{t+1}) \beta_{t+1}(j)}{\sum_k \alpha_T(k)}.$$

M-Step: Using $\gamma_t(i)$ and $\xi_t(i, j)$, we update:

$$\hat{A}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)},$$

$$\hat{\mu}_i = \frac{\sum_{t=1}^T \gamma_t(i) O_t}{\sum_{t=1}^T \gamma_t(i)}, \quad \hat{\Sigma}_i = \frac{\sum_{t=1}^T \gamma_t(i) (O_t - \mu_i)(O_t - \mu_i)^\top}{\sum_{t=1}^T \gamma_t(i)}.$$

We ensure numerical stability by performing computations in the log domain and applying regularization to covariance matrices. These considerations are reflected in the code, where computations use `torch.logsumexp` and small perturbations to maintain positive-definite covariances.

The model is iteratively trained until convergence criteria are met (e.g., no significant improvement in log-likelihood or a maximum number of iterations is reached).

4 Decoding with the Viterbi Algorithm

4.1 Objective

Once the HMM is trained, decoding aims to find the most likely state sequence given an observed sequence of features. We use the Viterbi algorithm, which efficiently searches the state space to produce the single best state path.

4.2 Algorithm

The Viterbi algorithm uses a similar recursion to the forward pass, but instead of summing over all previous states, it maximizes:

$$\delta_t(i) = \max_j [\delta_{t-1}(j) + \log A_{ji} + \log B_i(O_t)],$$

with initialization:

$$\delta_0(i) = \begin{cases} 0 & i = \text{start state} \\ -\infty & \text{otherwise} \end{cases}$$

A back-pointer array $\psi_t(i)$ records the state j that maximized the expression for $\delta_t(i)$, allowing backtracking to reconstruct the optimal path after processing all frames.

4.3 Implementation Considerations

All computations are performed in log-space for numerical stability. The transition model uses a strict left-to-right topology, and zero or near-zero probabilities are avoided via smoothing. After finalizing the forward passes, the best path is extracted by backtracking from the final time step to the start.

This decoding step identifies the most likely hidden state sequence, which can then be mapped to phonemes or words. The code leverages GPU-accelerated operations when available to ensure efficient inference.

4.4 Practical Considerations and Extensions

Beyond finding the most likely state sequence, the Viterbi algorithm also provides valuable insights into how the model aligns the observed speech frames with particular phonemic or word segments. For instance, by examining the back-pointers at each time step, we can pinpoint which segments of the acoustic signal are most responsible for driving the state transitions, helping us understand misclassifications or identify especially challenging phonetic distinctions.

In situations where multiple words are modeled by separate HMMs, the Viterbi algorithm can be applied to each candidate model, and the word whose model yields the highest final score is selected. This approach is straightforward, but can be extended to more complex tasks such as

continuous speech recognition. There, a network of word-level HMMs is combined into a large search graph, and the Viterbi algorithm is used to find the optimal word sequence. Although this increases computational complexity, techniques like beam search can reduce the search space by pruning unlikely paths early.

Finally, practical implementations often incorporate additional constraints or smoothing parameters to handle low-probability transitions, ensure numerical stability, and maintain reasonable decoding times. By carefully tuning these factors and leveraging efficient GPU-accelerated operations, the Viterbi algorithm remains a powerful and efficient tool for real-time speech recognition tasks.

5 Results

The training phase is a crucial component in developing robust speech recognition systems. During this phase, the Hidden Markov Models (HMMs) are optimized to accurately recognize isolated words within the target vocabulary. The Baum-Welch algorithm is employed to train the models, utilizing extracted Mel-Frequency Cepstral Coefficients (MFCCs) as input features. To ensure the models capture the diversity and variability of speech patterns effectively, the training data is carefully divided to balance representation across the eleven words in the vocabulary. The primary objective of this phase is to minimize the loss function, aiming for stable convergence that reflects the system's ability to generalize from training data to unseen data.

5.1 Training and Validation Results

Overview of Training and Validation. The dataset for this project was divided into training and validation subsets to ensure robust model development. Specifically, 70% of the audio samples were allocated for training, while the remaining 30% constituted the validation set. This validation data is distinct from test data and is used to monitor model performance during training, helping to fine-tune parameters and prevent overfitting or underfitting.

After extracting vocal tract features of each 11 target words represented by MFCCs, the coefficients are normalised to ensure that all features contribute equally, regardless of their varying original magnitude caused by the different energy and amplitude of the original audio. These features serve as the initial input to the HMMs.

The primary objective of the training process was to initialize and train HMMs for each of the 11 target words. The models were optimized using the Baum-Welch algorithm, aiming to accurately recognize isolated words. Performance was validated on unseen data (validation dataset), ensuring the system's ability to generalize beyond the training data.

Loss Curves. The loss function quantifies the difference between the model's predicted output and the actual target output. The loss curve indicates how well a machine learning model, here HMM, is learning during the training process, as evaluated at each step or iteration.

According to the loss curves, all 11 HMM models converge as the loss continues reducing as iteration increases till stabilizing at a certain level, indicating that the model generalizes well to unseen data. Both training and validation losses decrease significantly in the early epochs for every model while the gap between training and validation loss is small in the later epochs, which is desirable. This trend indicates that the model is performing effectively, learning the data patterns without overfitting.

Figure 3 shows the best results for training and validation loss across three words: "hud," "hard," and "hod." For "hud," there is minimal gap between the two curves, with the loss value being approximately 1800, indicating a good fit. On the other hand, "hard" and "hod" yield the worst results, with relatively high loss values and a large gap between the two curves. In "hard," the

validation loss is higher than the training loss, which may be caused by overfitting, where the model captures not only the underlying patterns but also noise and irrelevant details, or by insufficient training data. For "hod," the training loss is higher than the validation loss, which could be due to a small validation set.

Performance Metrics. The accuracy of word recognition in the validation process reaches 63.27% with detailed information concluded by a confusion matrix in Figure4, from which clearly indicates most words can be mostly categorized into the correct label instead of other words (diagonal entries are relatively higher than off-diagonal values). Due to lack of validation sets, 2 of the diagonal entries do not show much difference from their corresponding off diagonal values.

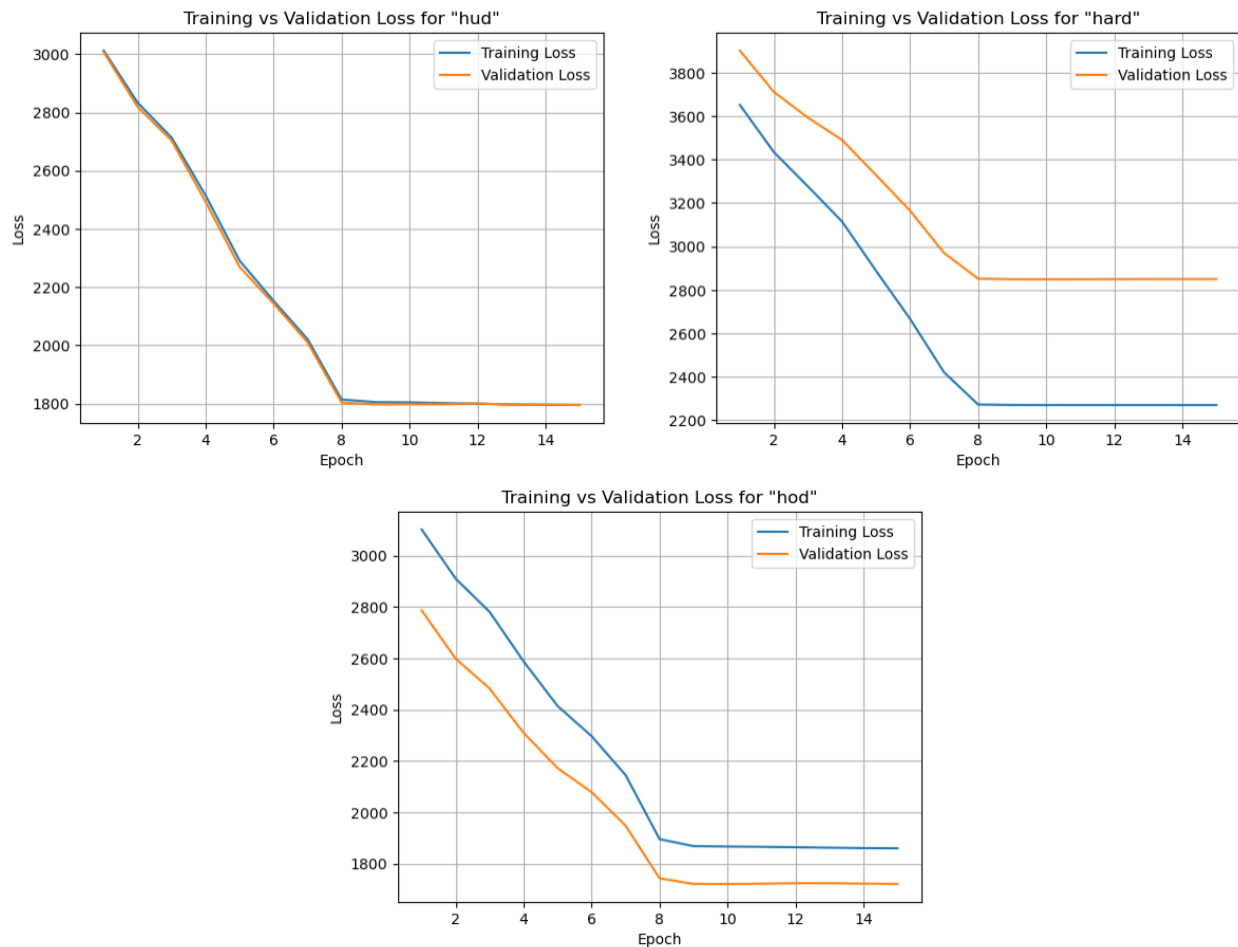


Figure 3: Training and Validation Loss Curves for "hud"(top left), "hard"(top right) and "hod"(bottom)

WER vs Number of States. Figure5 shows the how WER changes with different numbers of HMM states (6, 8, 10 states) and the best performance (lowest WER) occurs at 8 HMM states. Higher WER at 10 states could be caused by overfitting when the number of states exceeds the

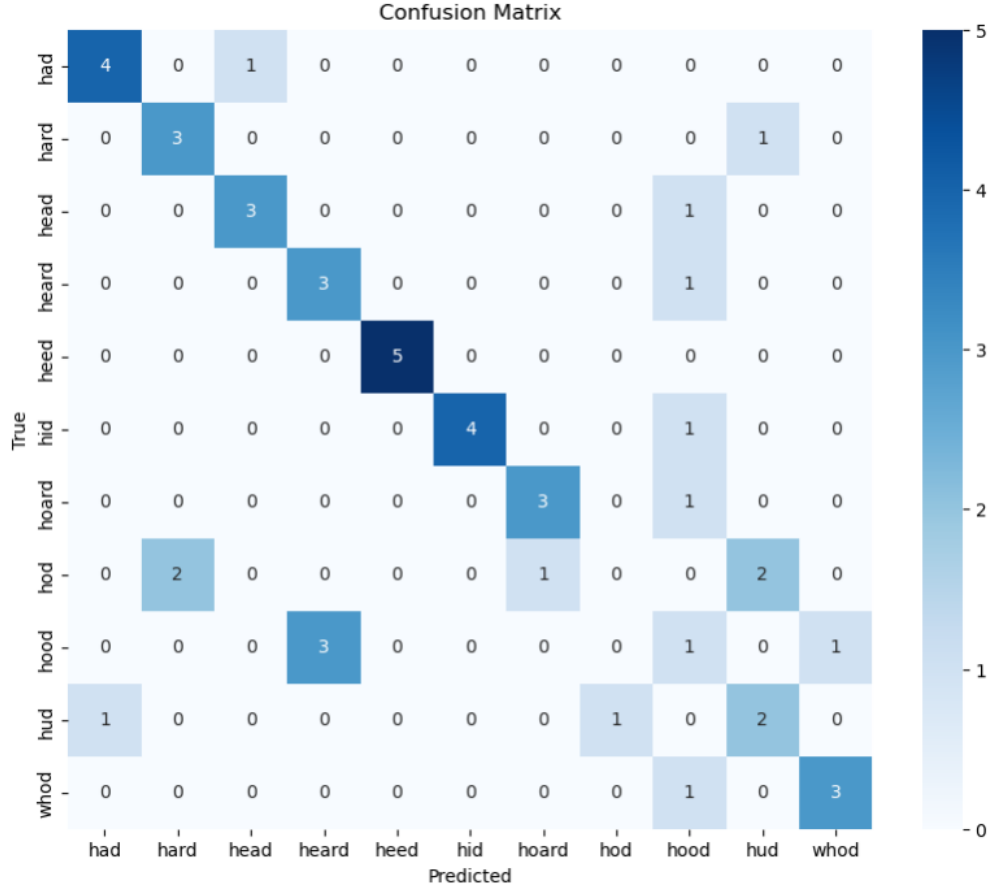


Figure 4: Confusion Matrix for Validation Result

complexity of the data. Higher WER at 6 states could be caused by underfitting where the model may not capture all necessary variations or structure.

State Duration Analysis. Normalized state duration plots visualize how long an HMM spends in each state, scaled relative to the total word duration. They help assess the model’s understanding of the temporal structure and consistency across different utterances. This analysis is valuable in evaluating and fine-tuning HMMs. Figure 6 indicates the normalised state durations for word “hood” where certain states (1, 2, and 5) dominate the word’s representation, while others (3, 4, and 6) play minor or no roles. The state that cost the longest duration could be the ‘oo’ part. However, the Zero duration at state 3 is kind of unique and may require further finetuning.

The completion of the training phase marked a significant milestone, as the models exhibited promising convergence patterns and reduced validation loss, especially for key words like “hud.” However, specific instances of overfitting and underfitting underscored the need for further refinement. With the training phase concluded, the focus shifts to evaluating the system’s performance on an entirely separate dataset. The subsequent section delves into the test results, analyzing the

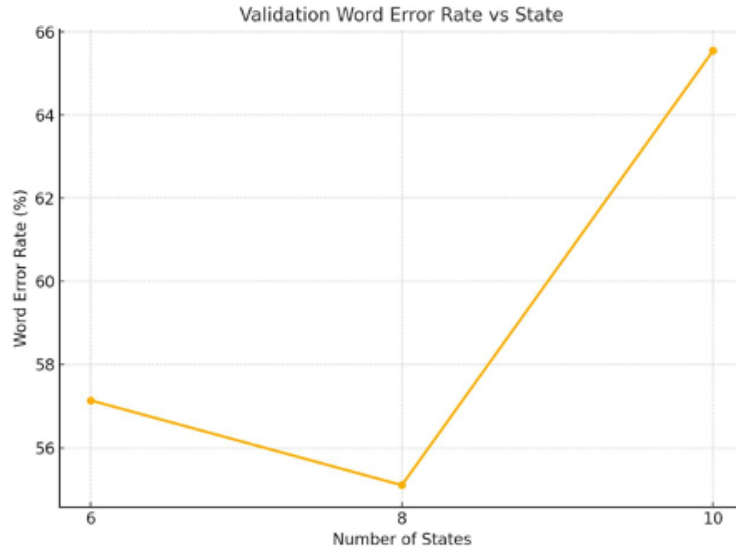


Figure 5: Word Error Rate of Validation Results under Different Number of HMM States

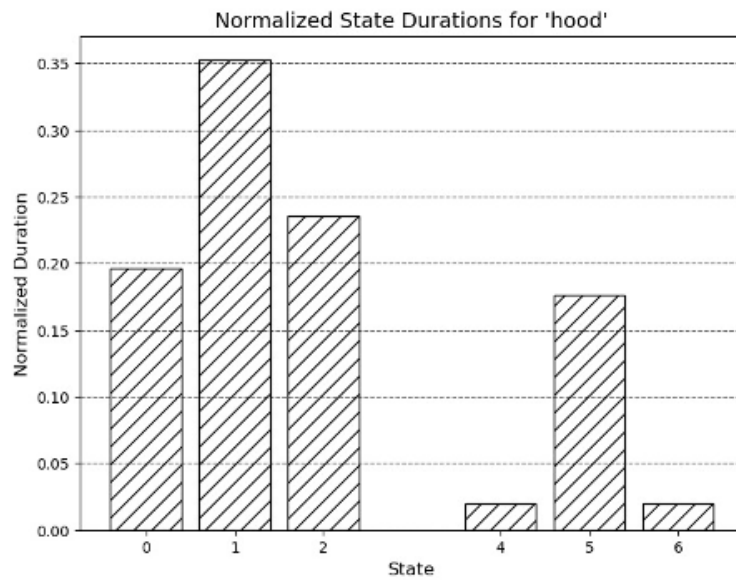


Figure 6: Normalized State Durations for “hood”

system’s ability to generalize to new data and providing insights into its real-world applicability. This transition from training to testing emphasizes the significance of assessing the robustness and scalability of the developed speech recognition models.

5.2 Test Results

Overview of Training and Validation The 8-state Hidden Markov Model (HMM) was tested on a new dataset comprising 11 distinct words that were not part of the training process. The primary goal was to assess the model's generalization ability and pattern recognition capabilities. It achieved a test accuracy of 53.64%, slightly below its validation accuracy of 55.10%. This performance highlights challenges such as overlapping feature spaces and the phonetic similarities among the test words.

Table 1: Comparison of Test and Validation Accuracy Across Models

Number of States	Validation Accuracy (%)	Test Accuracy (%)	Recognition Error Rate (%)
6	57.14	50.91	49.00
6	57.14	50.91	49.09
8	55.10	53.64	46.36
10	65.55	44.55	55.45

The table presents a comparison of the trade-offs between model complexity and generalization:

- The 8-state model strikes the best balance, achieving a test accuracy of 53.64% and a validation accuracy of 55.10%. This moderate generalization is reflected in a recognition error rate of 46.36%.
- The 6-state model exhibits a smaller gap between validation (57.14%) and test accuracy (50.91%), indicating underfitting due to its limited complexity.
- The 10-state model, with the highest validation accuracy (65.31%), suffers from overfitting. Its low-test accuracy (44.55%) and poor generalization further support this conclusion.

The 8-state model emerges as the most balanced option, offering a manageable trade-off between complexity and performance. Consequently, it is the preferred choice for achieving generalization.

Confusion Matrix (Test Data) The confusion matrix offers a comprehensive overview of the model's performance on the 11 unique test words. As illustrated in the figure, the model demonstrated high accuracy in correctly classifying instances of words like "hoard" and "hid," while it faced challenges with words like "hood," resulting in misclassifications.

Key Observations:

- The diagonal entries (true positives) indicate correct classifications, while off-diagonal values represent misclassifications.
- Misclassifications are prominent for phonetically similar words, such as "hood" being confused with "whood" and "heed" with "hid."

- Words like "hard" and "hod" show significant overlap, contributing to erroneous predictions.

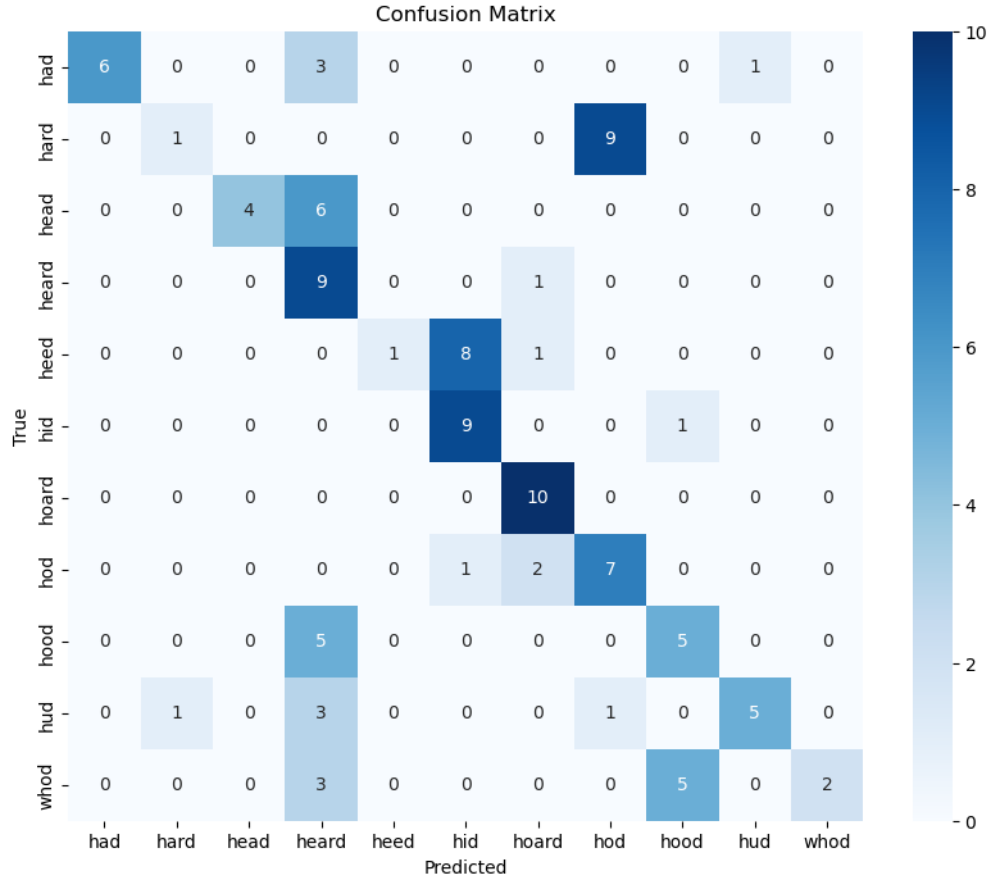


Figure 7: Confusion Matrix for 8 States HMM for Test data

Feature Space and Misclassification Analysis To enhance our comprehension of the confusion matrix's outcomes, we conducted feature space analysis and misclassification analysis. These analyses were designed to elucidate the reasons behind the frequent misclassification of certain words, with a particular emphasis on overlapping feature spaces and phonetic similarities.

The t-SNE plot (Figure 8) visualizes the high-dimensional feature space of the test data in two dimensions:

- Dimension 1 and Dimension 2 are abstract mathematical axes derived from the t-SNE algorithm. They preserve relative distances between points in the original high-dimensional feature space while reducing the dimensions for visualization.
- Words with distinct clusters (e.g., "hoard") have clearly separable features, leading to high classification accuracy.

- Overlapping clusters, such as "hood" and "whood," reveal feature similarity in the acoustic space, explaining frequent misclassifications between these words.

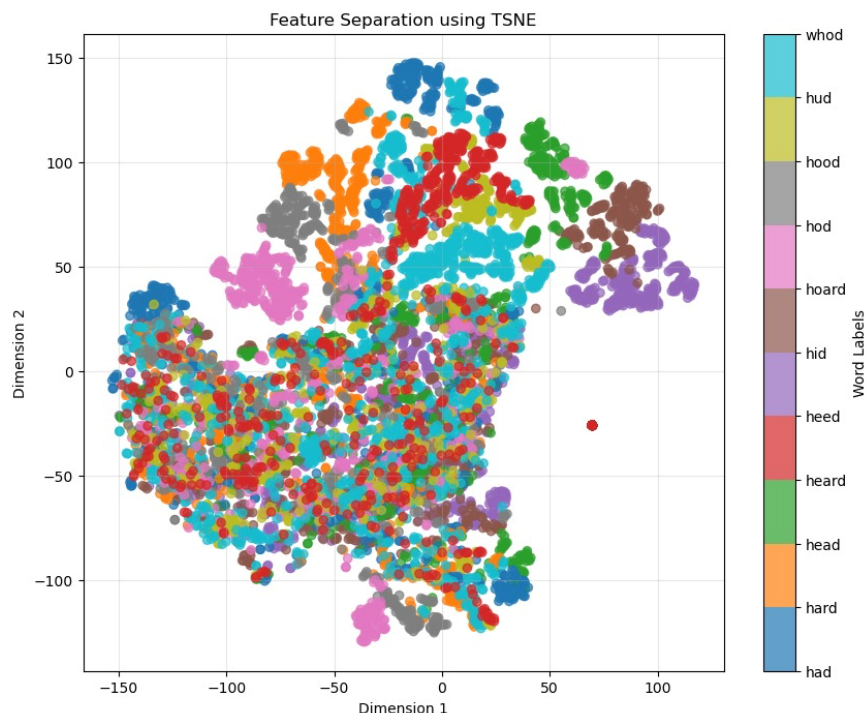


Figure 8: t-SNE Visualization of feature separation for the 11 distinct word classes in test data.

Misclassification Analysis Figure 8 further details the misclassification trends identified in the confusion matrix and feature space. The bar chart emphasizes the most frequent errors, which are primarily due to phonetic overlaps that lead to incorrect predictions.

Key Misclassifications:

1. "hood" and "whod" frequently merge due to their similar vowel patterns, as evidenced by their feature overlap in the t-SNE graph.
2. "Hard" and "hod" errors arise from the challenge in distinguishing the subtle acoustic differences between these words.

State Transition Analysis To delve deeper into the root causes of misclassifications and the variability in feature separation, we examined the state transitions within the Hidden Markov Models (HMMs). Below is the state transition matrix for the word 'hood', which illustrates the probabilities of transitioning between each state during decoding.

Figure 10 illustrates the most probable transitions between states. For instance, state 0 frequently transitions to itself (75%) or to state 1 (10%), indicating strong self-loops that ensure continuity

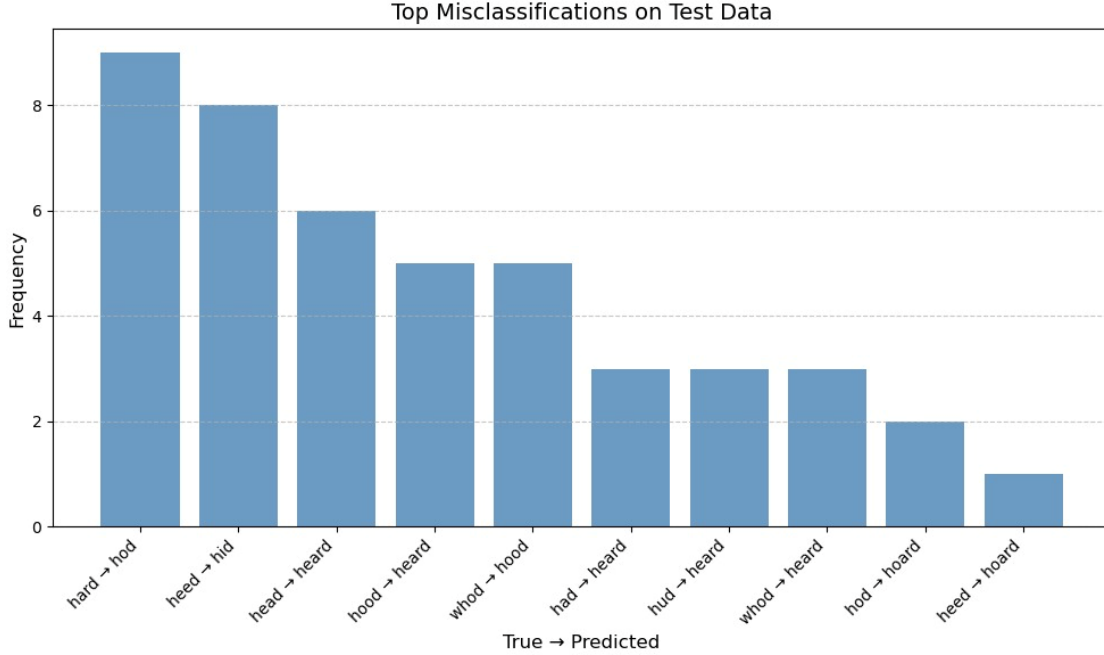


Figure 9: Top misclassifications on the test data, sorted by frequency.

in the signal representation. These self-loops are crucial for managing variations in speech duration. The off-diagonal values represent transitions to neighboring states, reflecting the gradual progression through phonetic features.

Log-Likelihood Analysis Building on the state transitions, we assessed the average log-likelihoods of test words across all models. The figure below presents the results, with higher (less negative) log-likelihoods indicating a better alignment with the corresponding Hidden Markov Model (HMM).

Figure11 demonstrates that certain words, such as **‘hard’** and **‘hid’**, consistently attain higher log-likelihoods because their feature spaces are well-separated and have fewer overlaps in phonetic features. On the other hand, words like **‘hood’** and **‘whod’** exhibit lower log-likelihoods, which aligns with the confusion matrix results and the overlapping feature spaces that were previously observed.

State Sequence Visualization The log-likelihood findings prompted a closer examination of individual test sequences. For instance, the figure below illustrates the state sequence for a test example of the word **‘hood’**.

Figure12 illustrates the model’s progression through states, showing how it decodes the audio sequence frame by frame. The consistent movement through states corresponds to the phonetic structure of **‘hood’** confirming the correspondence between transitions and the observed audio features.



Figure 10: State Transition Matrix for 'hood'

In essence, the 8-state HMM model successfully captured temporal patterns. However, challenges such as overlapping feature spaces and phonetic similarities, evident from the confusion matrix and misclassification analysis, underscore areas for enhancement. Consequently, Section 3.3 delves into a detailed error analysis to pinpoint and rectify these issues.

5.3 Error Analysis

In this section, we systematically analyze the sources of errors observed in the 8-state Hidden Markov Model (HMM) model. Our goal is to identify patterns in the misclassifications and understand their underlying causes, which will provide valuable insights for improving the model's performance.

Key Observations:

- I. Impact of Phonetic Similarities: Words like “hood” and “whoood,” or “heed” and “hid,” often overlap due to their phonetic similarities. This is because they share acoustic features, leading the model to misclassify test examples. The misclassification graph illustrates this

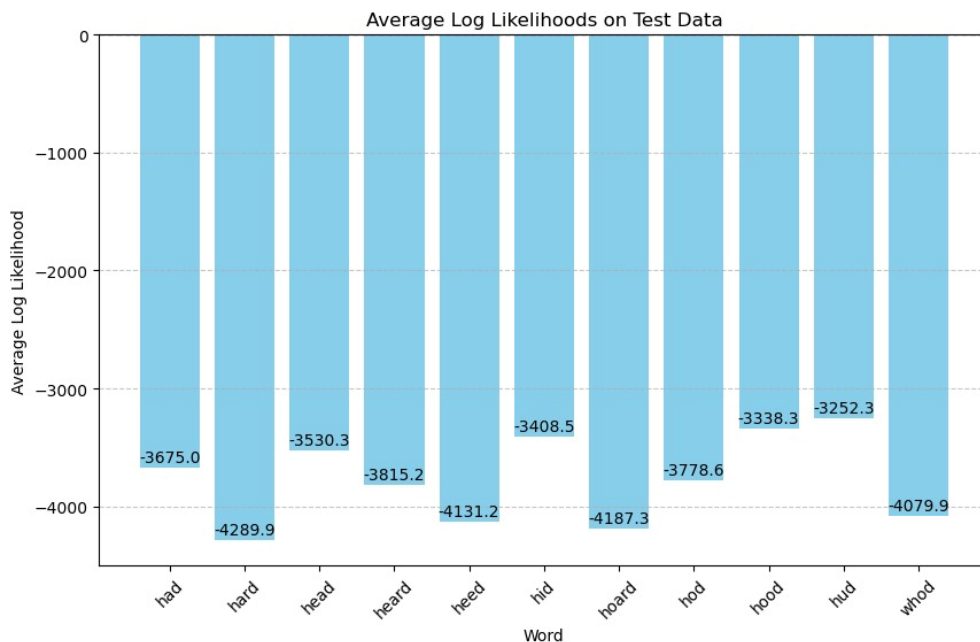


Figure 11: Average Log Likelihoods on Test Data

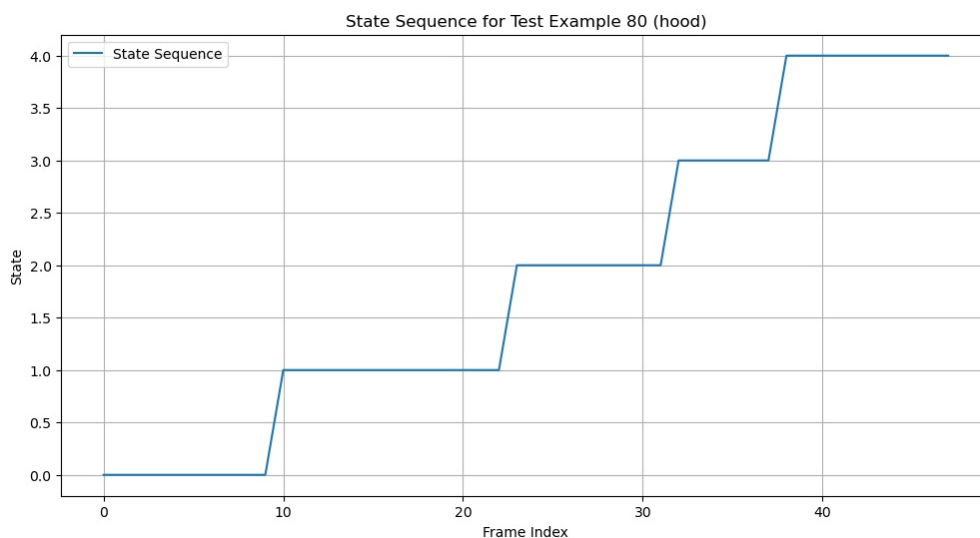


Figure 12: State Sequence for Test Example (hood)

phenomenon.

- II. Feature Space Overlap: Analysis of the feature separation graph reveals that clusters for words like “hood” and “whood” merge significantly. This confirms that overlapping feature representations contribute to errors. To address this, we need additional discriminative features or model enhancements to better separate these phonemes.

- III. State Transition Challenges: State transition matrices show that certain states were overused or underutilized. For instance, states in “hood” exhibited uneven transitions, suggesting insufficient training data for rare phoneme sequences.
- IV. Log Likelihood Disparities: Words with lower average log likelihoods, such as “heed” and “heard,” consistently have higher misclassification rates. This indicates that the model struggles to assign strong confidence scores to these words, possibly due to less distinctive patterns in their acoustic features.
- V. Dataset Limitations: The test set’s distribution shows a concentration of errors in words that were acoustically similar or had lower training samples. This highlights the need for a more balanced dataset with varied examples of each word.

The error analysis revealed critical limitations, such as challenges with phonetically similar words and feature space overlaps. However, it also highlighted the strengths of the 8-state HMM system. These findings underscore areas for improvement, including refining feature representations and enhancing the system’s robustness. The conclusion will summarize these findings and propose future directions to enhance the system’s accuracy and real-world applicability.

6 Conclusion

In this work, we developed a speech recognition system using Hidden Markov Models (HMMs) and Mel-Frequency Cepstral Coefficients (MFCCs). Our approach successfully recognized many spoken words and achieved a test accuracy of approximately 53.64%. While this performance indicates that the chosen methodology can handle a variety of speech signals, it also highlights areas where further improvements are needed.

Limitations

Despite its successes, the model faced several limitations. It struggled to differentiate words with similar pronunciation, indicating that the assumptions underlying HMMs may not fully capture the subtle acoustic differences between closely related sounds. The reliance on a limited vocabulary and training set reduced the system's adaptability to new words and diverse speaking styles. Additionally, background noise and varying recording conditions posed significant challenges, affecting overall accuracy.

On a theoretical level, HMMs assume that successive frames of speech are independent and that the current state depends only on the previous one. These simplifications overlook the complex temporal dependencies in natural speech. Moreover, the model's use of simple probability distributions (like mixtures of Gaussians) might not be rich enough to represent the complexity of human speech patterns.[6]

Future Work

To address these limitations, future efforts could involve exploring more advanced and flexible models. Incorporating deep neural networks or hybrid HMM-DNN systems[7] might better capture intricate acoustic patterns and long-range dependencies. Expanding the training data to include a wider range of accents, vocabularies, and recording conditions could improve robustness and adaptability. Adding preprocessing techniques to handle noise or employing more sophisticated feature representations may also help distinguish between phonetically similar words. Through these enhancements, the system has the potential to achieve more accurate and flexible speech recognition in a broader range of real-world scenarios.

Word Count: 4316

References

- [1] X. Yang and L. Sun. English speech recognition system based on hmm in. In *Conference on Communication Faculty*, 2009.
- [2] X. Huang, J. Baker, and R. Reddy. A historical perspective of speech recognition. *Communications of the ACM*, pages 94–103, 2014.
- [3] D. Divy and G. Ashutosh. Contrast between simple and complex classification algorithms. *Statistical Modeling in Machine Learning*, pages 93–110, 2023.
- [4] B. Shannon and K. Paliwal. A comparative study of filter bank spacing for speech recognition. In *Microelectronic Engineering Research Conference*, 2003.
- [5] Anon. Librosa: Audio and music processing in python, 2024. Version 0.10.2, [Online]. Available: <https://librosa.org>.
- [6] L.R. Rabiner. A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [7] Longfei Li, Yong Zhao, Dongmei Jiang, Yanning Zhang, Fengna Wang, Isabel Gonzalez, Enescu Valentin, and Hichem Sahli. Hybrid deep neural network–hidden markov model (dnn-hmm) based speech emotion recognition. In *2013 Humaine Association Conference on Affective Computing and Intelligent Interaction*, pages 312–317, 2013.
- [8] S. S. R. Shyam and S. R. M. Prasanna. Speech recognition using hidden markov model with mfcc-subband technique. In *Proceedings of the International Conference on Signal Acquisition and Processing (ICSAP)*, pages 89–93, Bangalore, India, 2010.
- [9] Y. Zhang. The research of application of hidden markov model in the speech recognition. In *Proceedings of the 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)*, pages 104–107, Nanchang, China, 2021.
- [10] T. T. Cai and R. Ma. Theoretical foundations of t-sne for visualizing high-dimensional clustered data. *Journal of Machine Learning Research*, 23(228):1–54, 2022.
- [11] J. Pohle, R. Langrock, F. M. van Beest, and N. M. Schmidt. Selecting the number of states in hidden markov models: Pitfalls, practical challenges, and pragmatic solutions. *Statistical Science*, 32(4):611–634, 2017.